



GPS L2C SDR



University of Colorado
Boulder



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

Introduction (1/8)

GPS signal parameters

Signals		Code Freq.	Code Len.	Nav. Data		Modulation	Freq.	Band Width	Min.
		[Mcps]	[chip]	Type	D(t)/d(t)[bps]		[MHz]	[MHz]	Received Power[dBW]
L1 C/A		1.023	1023	NAV	50/50	BPSK-R(1)	1572.42	2.046	-158.5
L1 P(Y)		10.23	7 days	NAV	50/50	BPSK-R(10)		20.46	-161.5
L1 M		5.115	Unknown	MNAV	Unknown	BOC(10,5)		30.69	Unknown
L1C	L1 CD	1.023	10230	CNAV-2	50/100	BOC(1,1)		4.092	-157.0
	L1 CP	1.023	10230×1800	Pilot Channel		TMBOC(6,1,4/33)		14.332	
L2 P(Y)		10.23	7 days	NAV	50/50	BPSK-R(1)	1227.6	20.46	-161.5
L2C	CM	0.5115×2	10230	CNAV	25/50	BPSK-R(1)		2.046	-160.0
	CL		767250	Pilot Channel					
L2M		5.115	Unknown	MNAV	Unknown	BOC(10,5)		30.69	Unknown
L5C	L5I	10.23	10230×10	CNAV	50/100	QPSK-R(10)	1176.45	20.46	-157.9
	L5Q	10.23	10230×20	Pilot Channel				20.46	

Introduction (2/8)

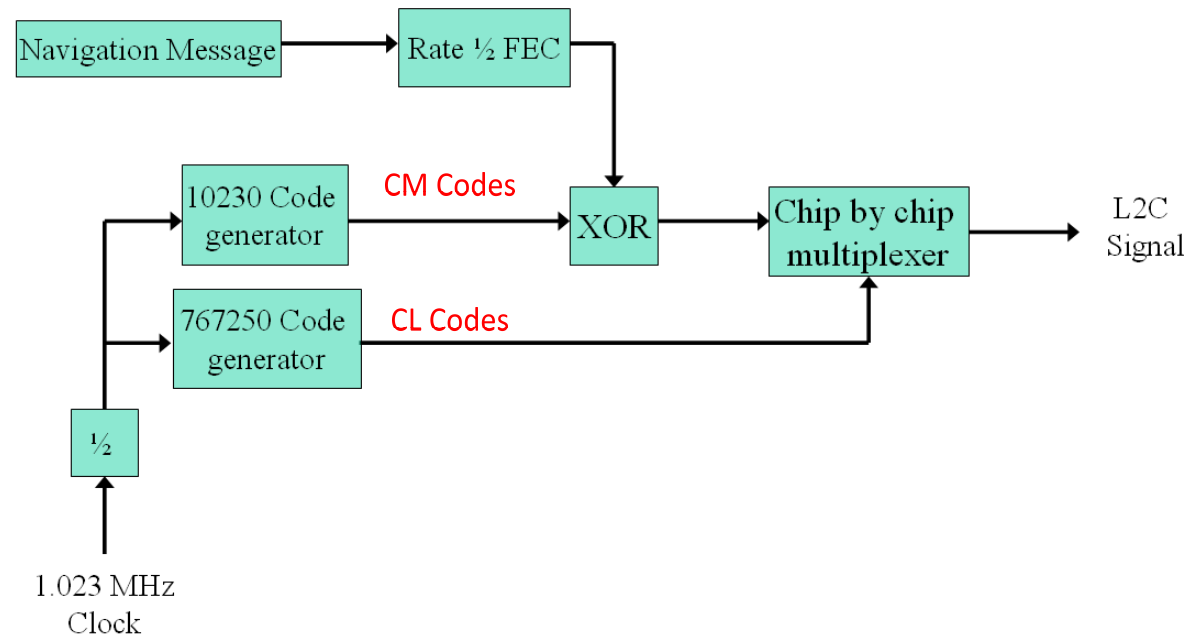
List of SVs transmitting L2C (*)

	Satellite	Launch (UTC)	Launch site	Block	No	SVN	PRN	Slot
19 Svs	USA-266	2/5/2016	CCAFS SLC-41	IIF	12	70	32	F5
	USA-265	10/31/2015	CCAFS SLC-41	IIF	11	73	10	E6
	USA-183	9/26/2005	CCAFS SLC-17A	IIRM	1	53	17	C4
	USA-190	9/25/2006	CCAFS SLC-17A	IIRM	2	52	31	A2
	USA-192	11/17/2006	CCAFS SLC-17A	IIRM	3	58	12	B4
	USA-196	10/17/2007	CCAFS SLC-17A	IIRM	4	55	15	F2
	USA-199	12/20/2007	CCAFS SLC-17A	IIRM	5	57	29	C1
	USA-201	3/15/2008	CCAFS SLC-17A	IIRM	6	48	7	A4
	USA-206	8/17/2009	CCAFS SLC-17A	IIRM	8	50	5	E3
	USA-213	5/28/2010	CCAFS SLC-37B	IIF	1	62	25	B2
	USA-232	7/16/2011	CCAFS SLC-37B	IIF	2	63	1	D2
	USA-239	10/4/2012	CCAFS SLC-37B	IIF	3	65	24	A1
	USA-242	5/15/2013	CCAFS SLC-41	IIF	4	66	27	C2
	USA-248	2/21/2014	CCAFS SLC-37B	IIF	5	64	30	A3
	USA-251	5/17/2014	CCAFS SLC-37B	IIF	6	67	6	D4
	USA-256	8/2/2014	CCAFS SLC-41	IIF	7	68	9	F3
	USA-258	10/29/2014	CCAFS SLC-41	IIF	8	69	3	E1
	USA-260	3/25/2015	CCAFS SLC-37B	IIF	9	71	26	B5
	USA-262	7/15/2015	CCAFS SLC-41	IIF	10	72	8	C5

(*) Wikipedia: https://en.wikipedia.org/wiki/List_of_GPS_satellites

Introduction (3/8)

Modulation scheme for the GPS L2 Signals

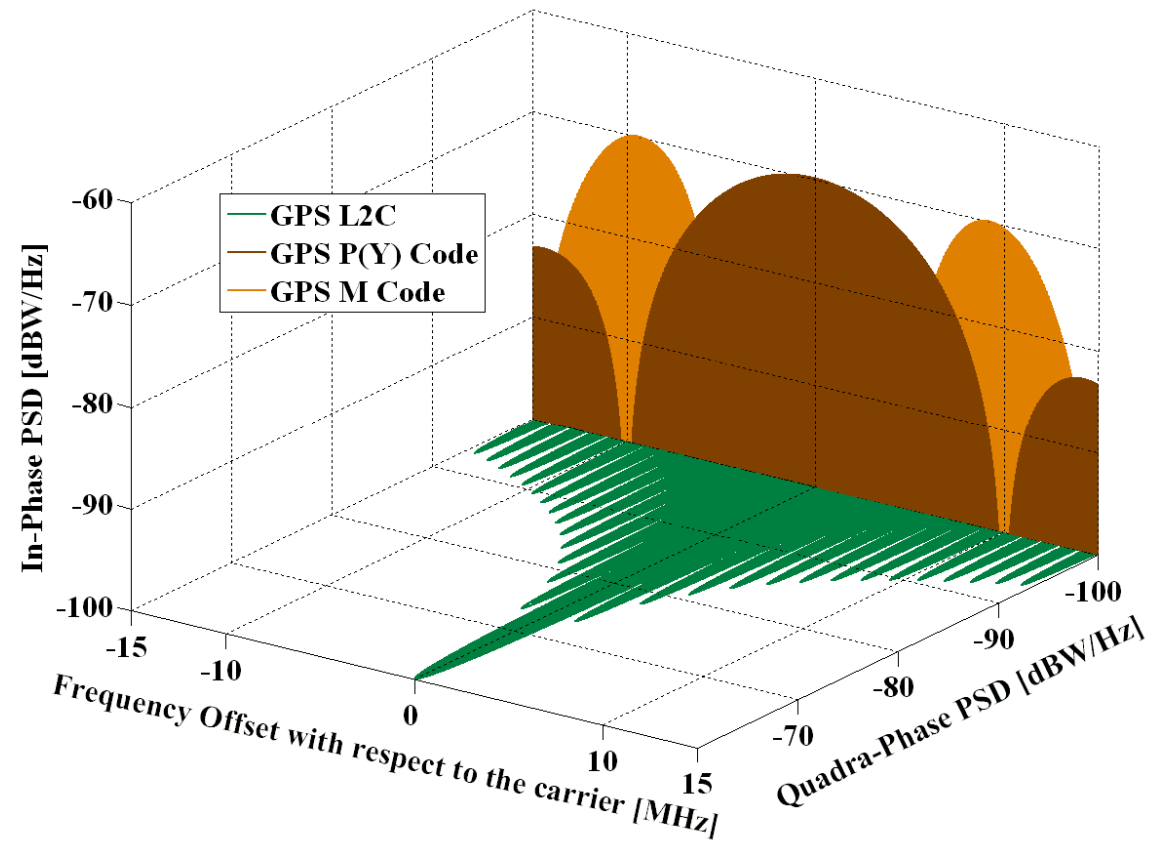


CM/CL Codes:

- ◆ Time multiplexed
- ◆ Chipping rate is doubled
- ◆ Synchronized with the X1 epochs of the P-code.

Introduction (4/8)

Spectra of the GPS Signals in L2 (*)



(*) Wikipedia: http://www.navipedia.net/index.php/GPS_Signal_Plan

Introduction (5/8)

L2 CM/CL code timing relationship

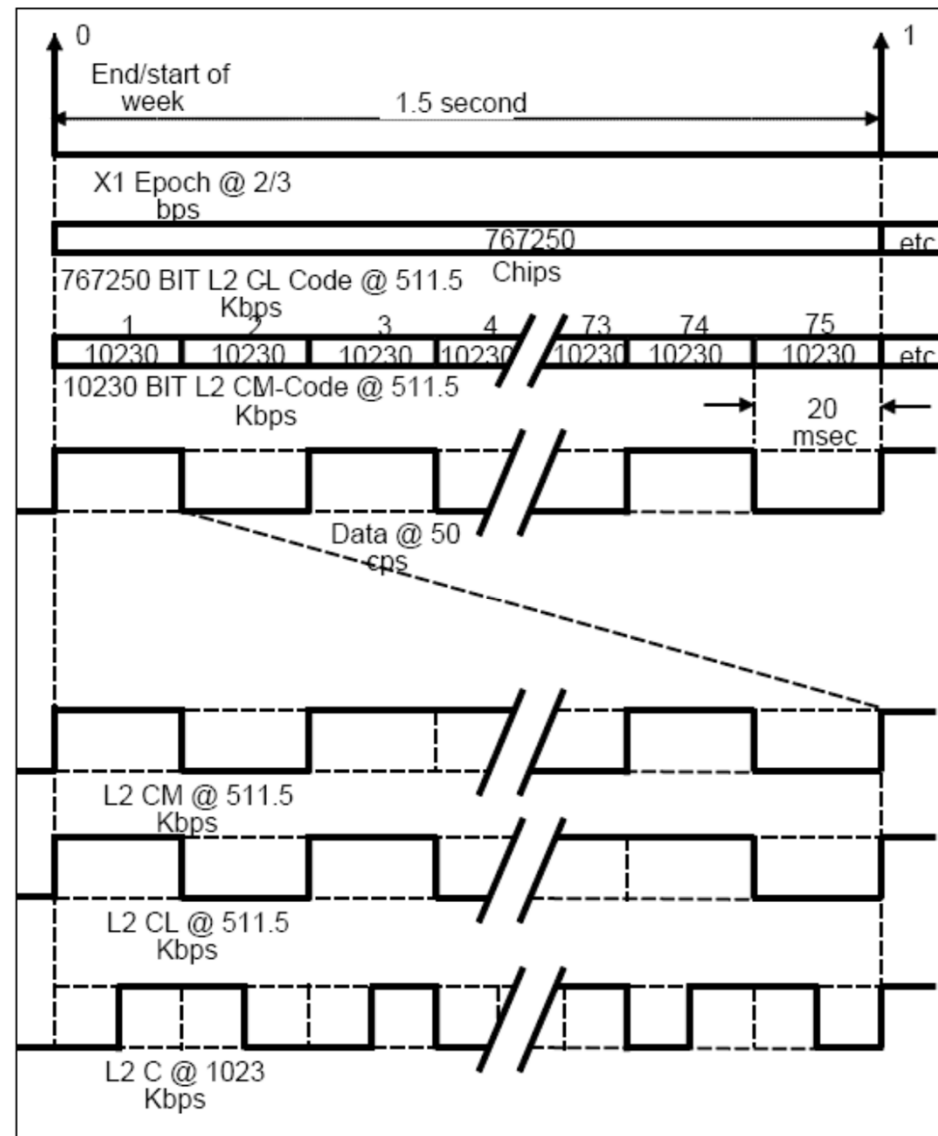


Fig. 3-12 of IS-GPS-200H

Figure 3-12. L2 CM-/L2 CL-Code Timing Relationships

Introduction (6/8)

CNAV message

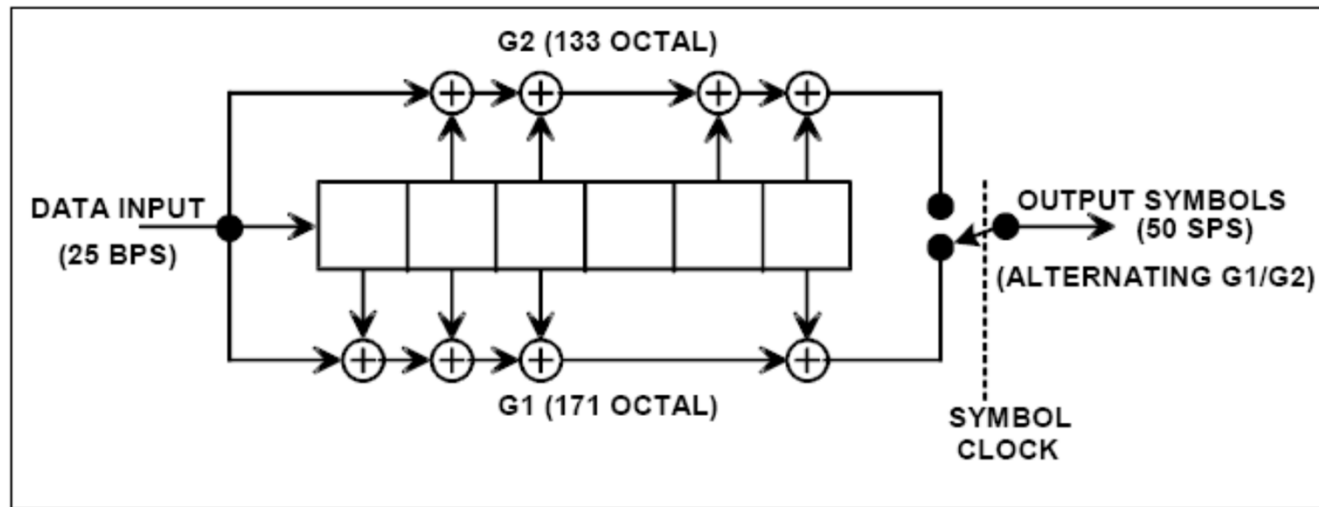


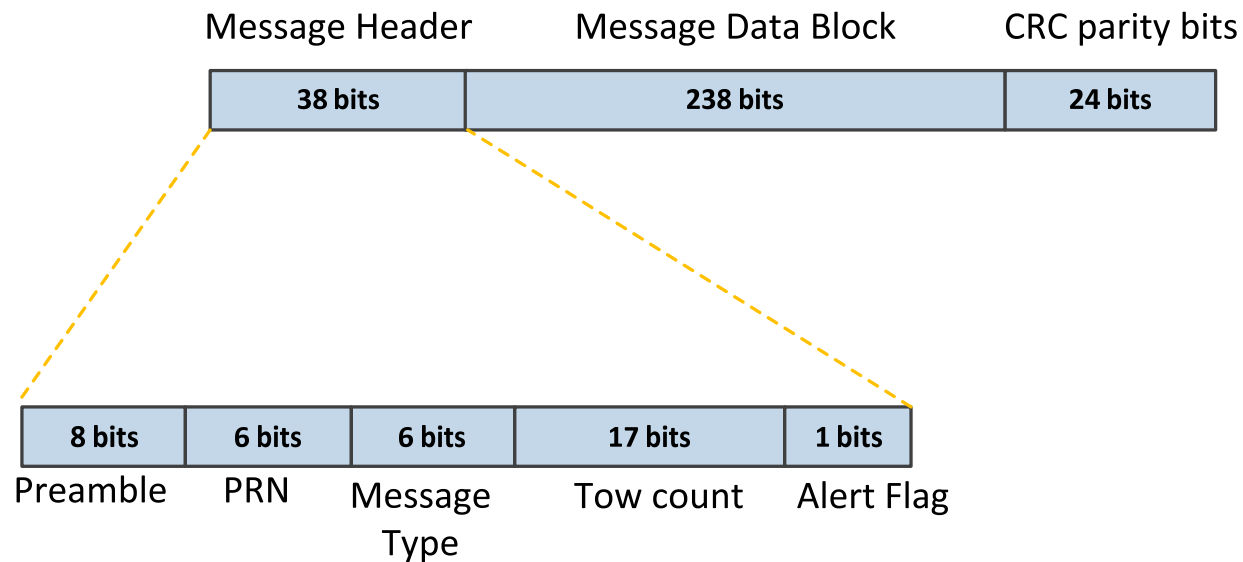
Fig. 3-14 of IS-GPS-200H

- ◆ $\frac{1}{2}$ fixed convolutional code + CRC-24Q
- ◆ **Viterbi Decoder** / Fano Decoder
- ◆ Same as Galileo I/NAV

Introduction (7/8)

Message structure of CNAV

300 bits long CNAV message



- ◆ Forward error correction (FEC) encoded by 1/2 convolutional code.
- ◆ Transmit a complete ephemeris every 48 seconds.

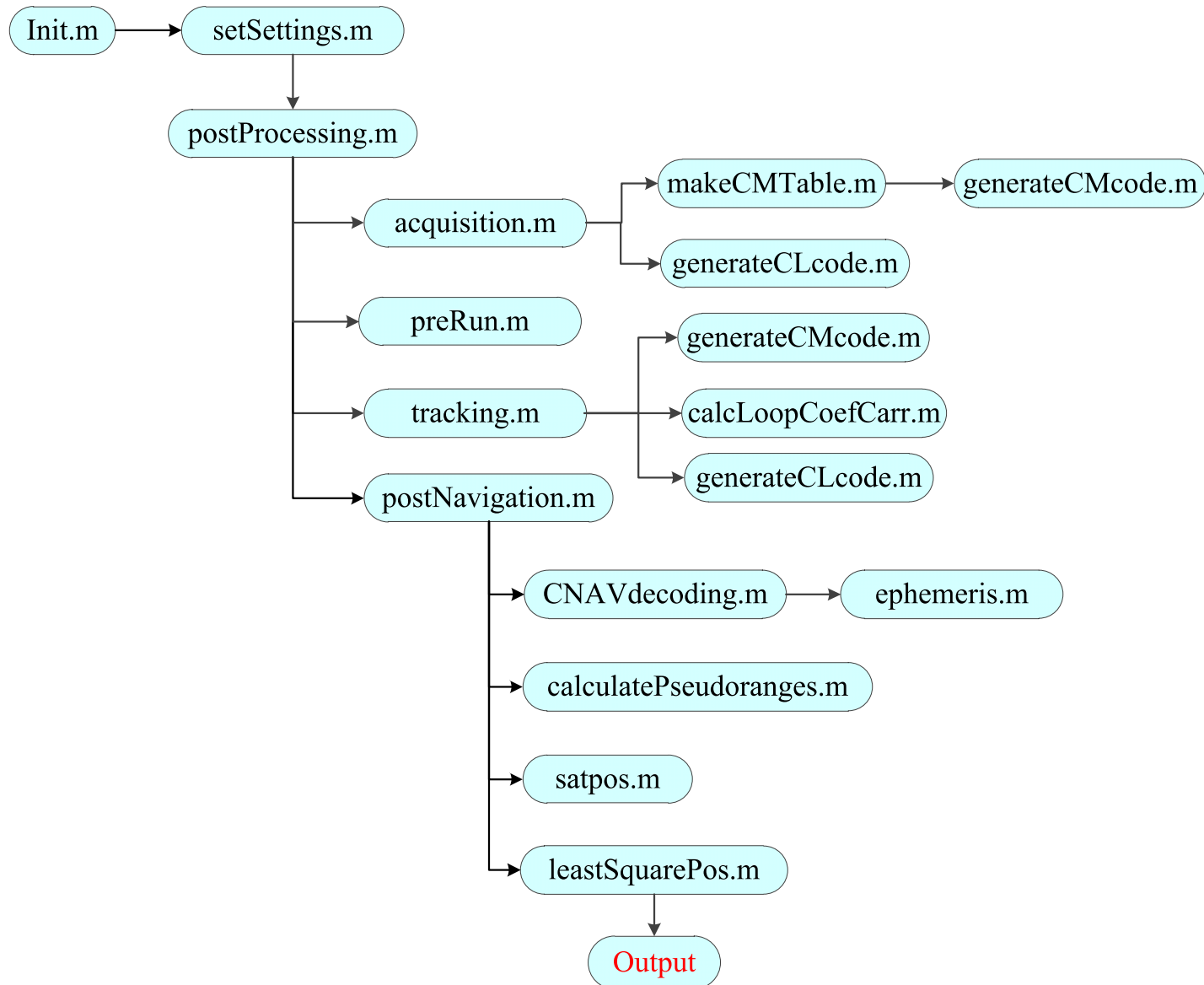
Introduction (8/8)

CNAV Message contents

- Requisite for receiver fix:
- Type 10
 - Type 11
 - Any one of 30-37

Type No.	Message Content	Max. Broadcast Intervals
10	Ephemeris	48 seconds
11	Ephemeris	48 seconds
30-37	Clock	48 seconds
30	Inter-Signal Correction and Ionospheric Data	288 seconds
33	UTC	288 seconds
35	GPS/GNSS Time Offset	288 seconds
31 or 12	Reduced Almanac	20 minutes
34 or 13	Differential Correction	30 minutes
32	Earth Orientation Parameters	30 minutes
14	Differential Correction	30 minutes
37	Midi Almanac	2 hours
36 or 15	Text Message	Not determined

SDR processing procedure



initSettings.m (1/2)

➤ Acquisition settings

- Single sideband for carrier freq. search
 - `settings.acqSearchBand = 5000;`
- Coherent integration time for acquisition can be set to 0 -20 (ms) by users
 - `settings.acqCohT = 20;`
- Carrier freq. search step
 - `settings.acqStep = 1000/settings.acqCohT/2;`
- Sampling rate threshold for downsampling
 - If input signal sampling freq. is too high for FFT computation, it can be resampled to a lower freq. to save processing time.
 - `settings.resamplingThreshold = 6e6;`
- Enable/disable use of downsampling for acquisition
 - `settings.resamplingflag = 1;      % 0 – Off, 1 - On`

initSettings.m (2/2)

➤ Tracking loops settings

- PLL and DLL integration time equals the CM code length

- `settings.intTime = 0.02;`

- Enable/disable use of **pilot channel** for tracking.

- `settings.pilotTRKflag = 1;` % 0 – Off 1 - On

➤ Code parameter setting

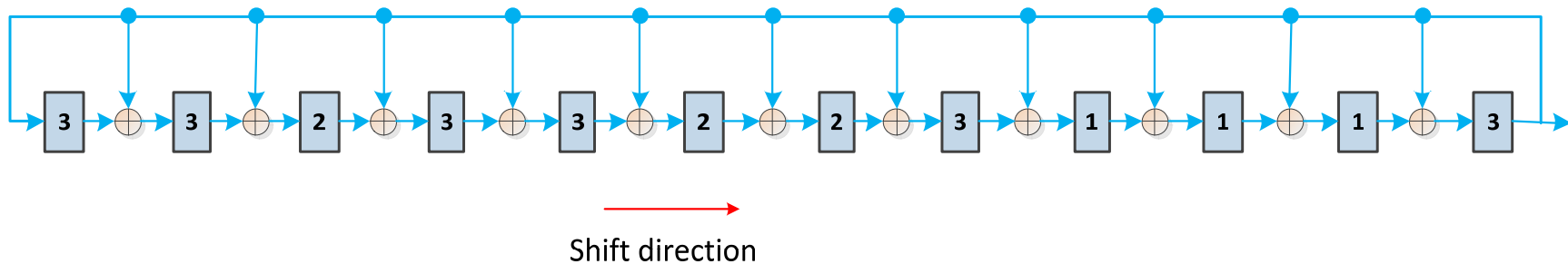
- L2 CM code rate is 0.5115 Mcps, and the code length is 10230 chips.

- `settings.codeLength = 10230;`

- `settings.codeFreqBasis = 0.5115e6;`

generateCMcode.m (1/3)

Characteristics of CM code



- ◆ The maximal polynomial is 1112225171 (octal) of degree 27.
- ◆ Initialized at the P-coder X1 epoch.
- ◆ PRN is assigned to PRN #1-63 and #159-210.
- ◆ Short cycled every count period of 10,230

generateCMcode.m (2/3)

➤ Generating procedure

- (1) Set generating polynomial

- `RegPos = [4 7 9 12 15 17 19 22 23 24 25];`

- (2) Initial-state table (See “IS-GPS-200H” for more details)

- `l2cm_init = [...];`

- (3) Select initial states for PRN

- `if (PRN >= 1 && PRN <= 63)`

- `shiftPos = PRN;`

- `elseif (PRN >= 159 && PRN <= 210)`

- `shiftPos = PRN - 95;`

- `elseif ( PRN < 1 || PRN > 210 || (PRN >= 64 && PRN <= 158) )`

- `disp(' PRN does not exist! ');`

- `return;`

- `End`

- `code_init = l2cm_init(shiftPos);`

generateCMcode.m (3/3)

➤ Generating procedure

- (4) Covert octal format to binary bit and fill in shift register

```
· reg = dec2bin(oct2dec(code_init)) - 48;  
· reg = [zeros(1,27-length(reg)) reg];
```

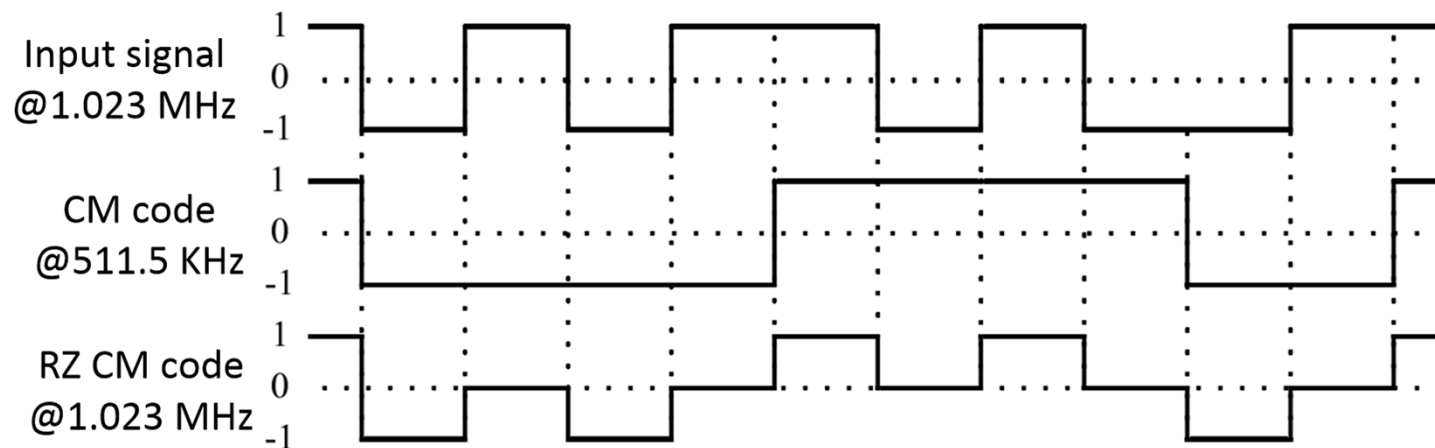
- (5) Generate CM code

```
· CMcode = zeros(1,CodeLength);  
· for index = 1:CodeLength  
·     CMcode(index) = reg(end);  
·     reg = circshift(reg',1)';  
·     reg(RegPos) = reg(RegPos) * CMcode(index);  
· end
```

makeCMTable.m (1/1)

➤ Return zero (RZ) CM code are generated

- `cmCode = generateCMcode(PRN,settings);`
- `cmCode = [cmCode;zeros(1,length(cmCode))];`
- `cmCode = reshape(cmCode,1,[]);`

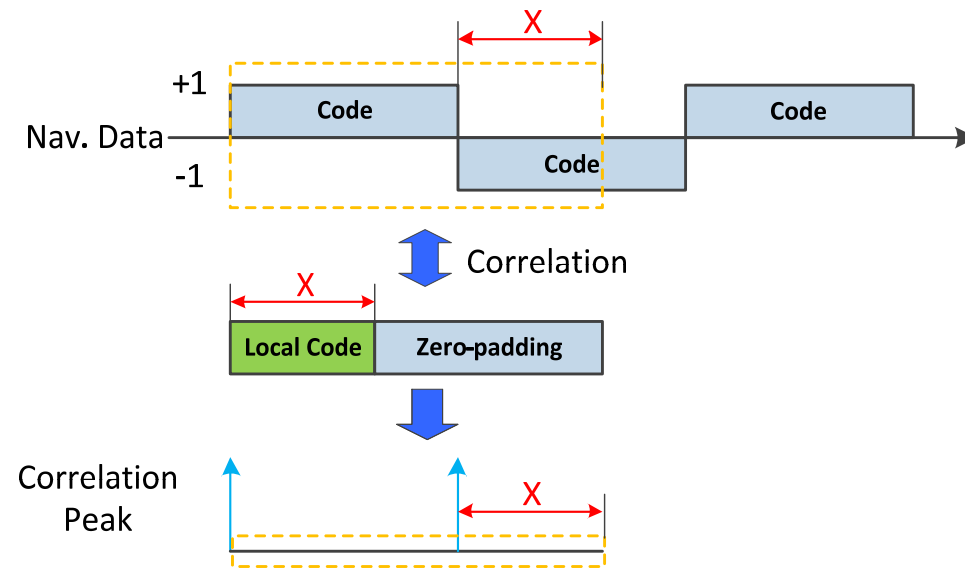


acquisition.m (1/16)

➤ Acquisition Method

- Circular correlation by **Zero-padding FFT**
- Perfect correlation can be obtained even when navigation bit is changed.
- Computation is doubled.

settings.acqCohT = X;

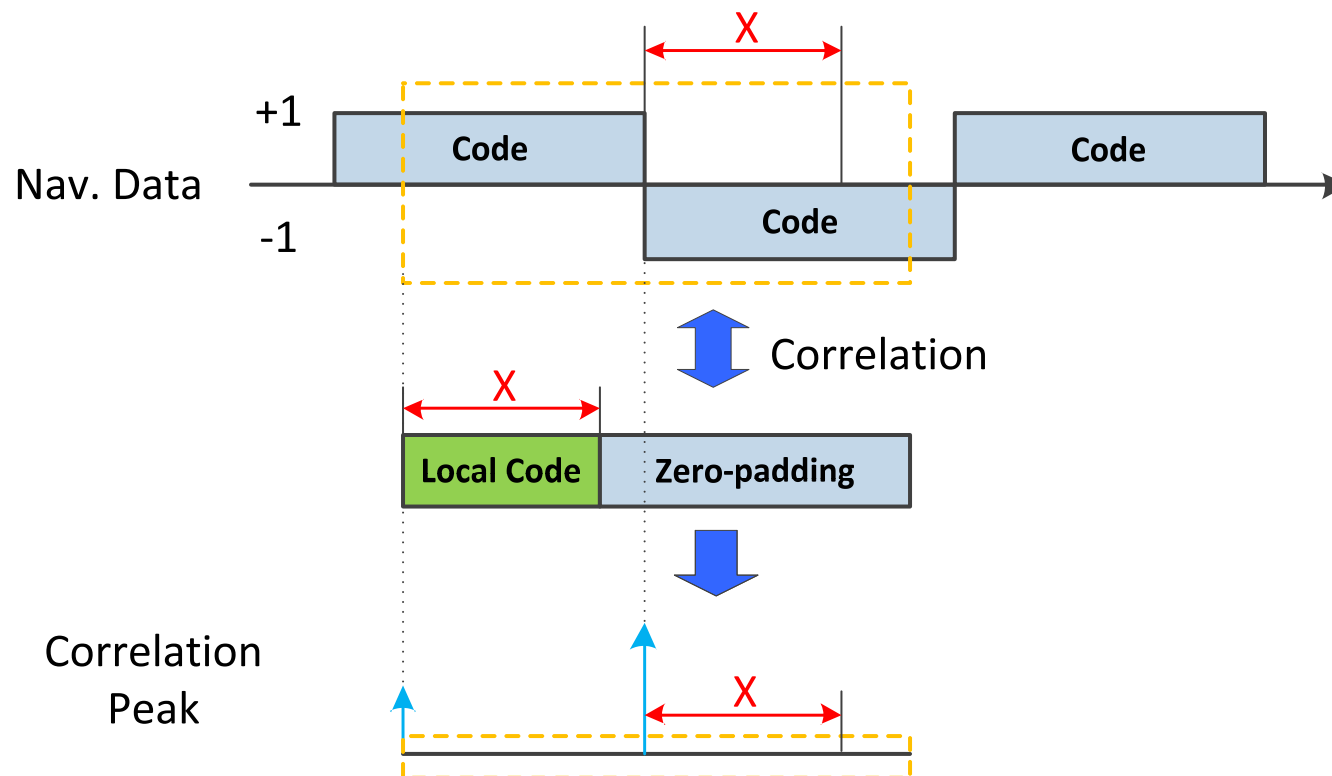


Case 1

acquisition.m (2/16)

➤ Acquisition Method

- Circular correlation by **Zero-padding FFT**

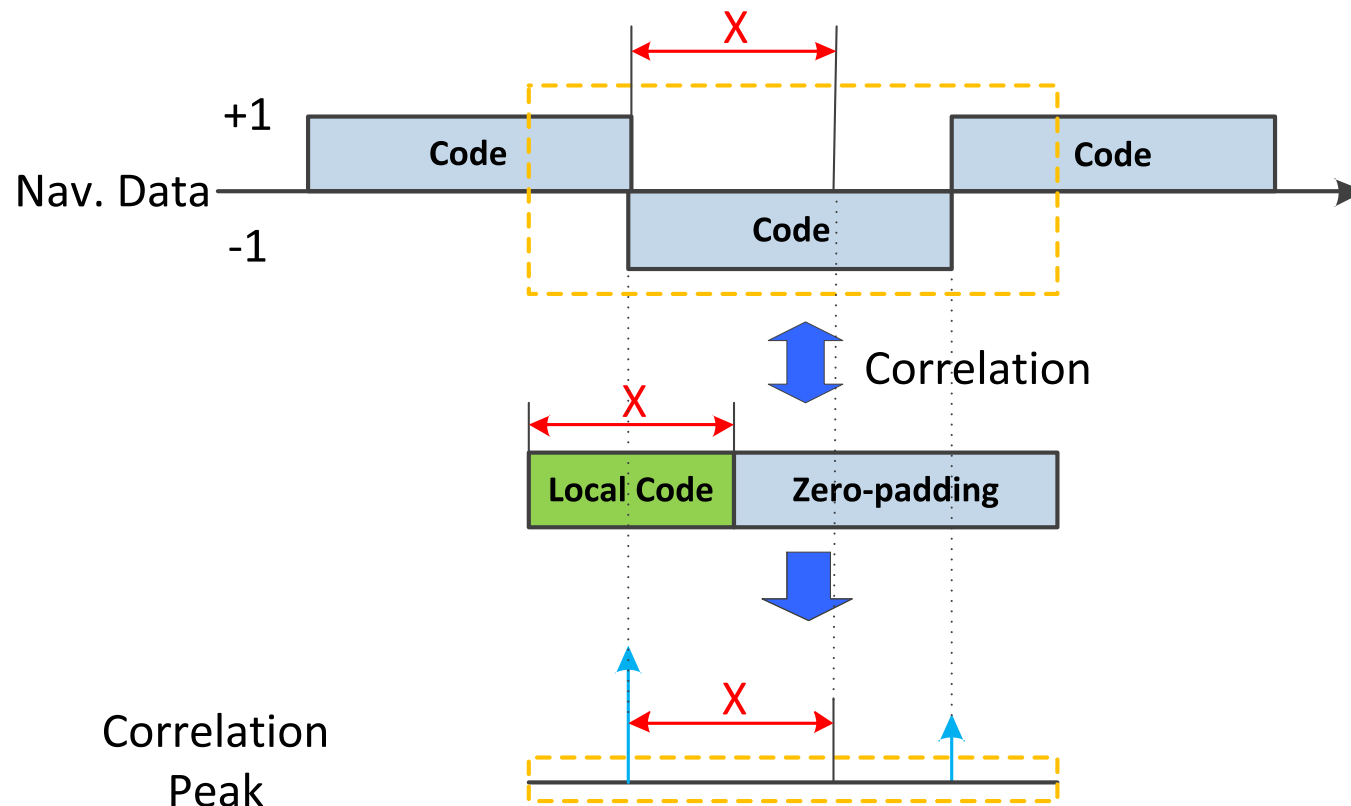


Case 2

acquisition.m (3/16)

➤ Acquisition Method

- Circular correlation by **Zero-padding FFT**

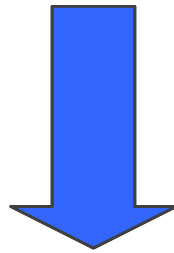


Case 3

acquisition.m (4/16)

➤ Selectable downsampling

- Computational load
 - `setsettings.acqCohT = 20;`
 - `setsettings.samplingFreq = 53e6;` → **2120000-point FFT**
- Computers crash all the time with MATLAB.
- **How to** settle this problem?



Downsampling
How?

acquisition.m (5/16)

➤ Bandpass sampling theory

- Assume that a real single-sideband is located at $[-f_u, -f_l] \cup [f_l, f_u]$, where f_l and f_u are given by $f_l = f_0 - B/2$, $f_u = f_0 + B/2$, B is the information bandwidth, and f_0 is the center of frequency of the information band. To avoid harmful signal spectral folding in reconstructing the signal, acceptable sampling rate should be in the range of

$$\frac{2f_u}{n} < f_s < \frac{2f_l}{n-1}$$

$n = \left\lfloor \frac{f_u}{B} \right\rfloor$ denoting the largest integer less than or equal to $\frac{f_u}{B}$.

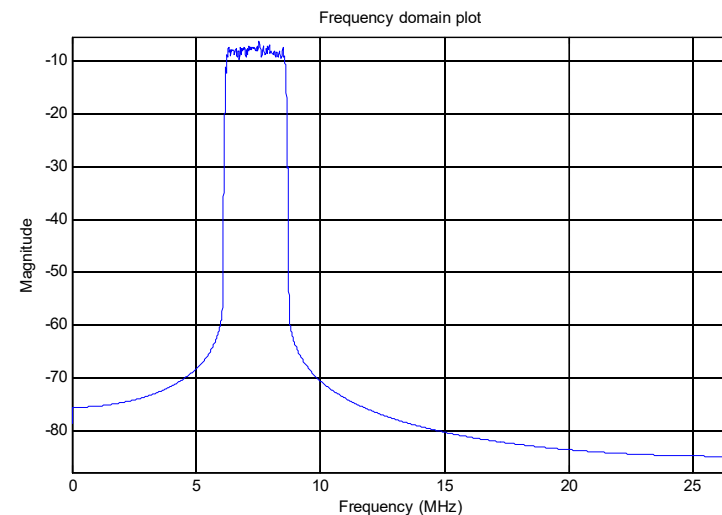
acquisition.m (6/16)

➤ Acquisition implementation

- Selectable downsampling

- (1) Filter out signal power outside the main lobe of CM code to avoid spectral folding

```
· fs = settings.samplingFreq;  
· IF = settings.IF;  
· BW = 1.2e6*2;  
· w1 = (IF)-BW/2;  
· w2 = (IF)+BW/2;  
· wp = [w1*2/fs-0.002 w2*2/fs+0.002];  
· b = fir1(700,wp);  
· longSignal = filtfilt(b,1,longSignal);
```



Power spectral density after
digital bandpass filtering

acquisition.m (7/16)

➤ Acquisition implementation

- Selectable downsampling

- (2) Calculate upper and lower boundary of the acceptable sampling

Freq. range:

- `fu = settings.IF + BW/2;`
- `n = floor(fu/BW);`
- `if (n<1)`
- `n = 1;`
- `end`
- `lowerFreq = 2*fu/n;`
- `fl = settings.IF - BW/2;`
- `if(n>1)`
- `upperFreq = 2*fl/(n-1);`
- `else`
- `upperFreq = lowerFreq;`
- `end`

acquisition.m (8/16)

➤ Acquisition implementation

- Selectable downsampling

(3) Find downsampling frequency

- Take the center of the acceptable sampling Freq. range as downsampling frequency.

- `settings.samplingFreq = ceil((lowerFreq + upperFreq)/2);`

(4) Downsample input IF signals

- `signalLen = floor((length(longSignal)-1) /oldFreq * settings.samplingFreq);`
- `index = ceil((0:signalLen-1)/settings.samplingFreq * oldFreq);`
- `index(1) = 1;`
- `longSignal = longSignal(index);`

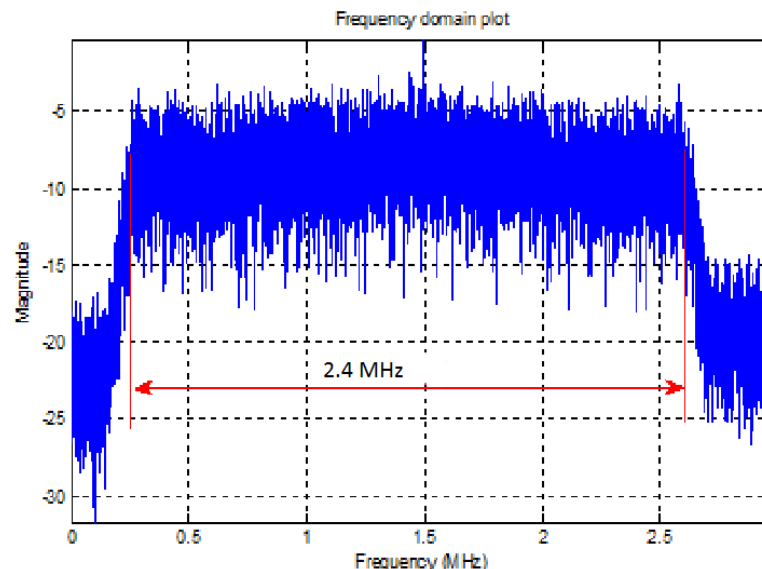
acquisition.m (9/16)

➤ Acquisition implementation

- Selectable downsampling

(5) Equivalent IF after downsampling

- Resampling is equivalent to down-converting the original IF by integer times of resampling freq.
- `settings.IF = rem(settings.IF, settings.samplingFreq);`



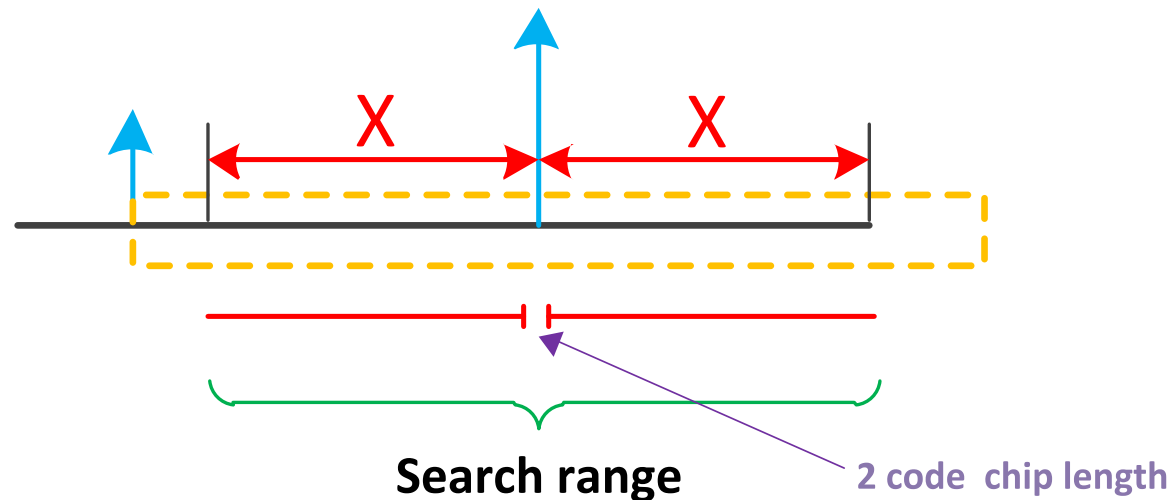
Power spectral density after downsampling

acquisition.m (10/16)

➤ Acquisition implementation

- Coarse acquisition

- Finding the second largest peak is a little different from the L1 C/A SDR.
- ...
- `codePhaseRange = [leftRange rightRange];`
- ...



acquisition.m (11/16)

➤ Acquisition implementation

- Fine acquisition

- Signal length (**findLen**) used to do fine acquisition depends on **settings.acqCohT**.
- **if(settings.acqCohT <=10)**
- **findLen = samples10msLength;**
- **else**
- **findLen = samplesXmsLen;**
- **end**

acquisition.m (12/16)

➤ Acquisition implementation

- Search the CL code phase

- CL code length is 75 times the length of CM code, so the CL code phase has a range of 1:75
- **for ind = 1:75**
- **% Make the digitized version of the CL code**
- **CLCodeSample = CLCode(codeValueIndex + settings.codeLength *(ind-1));**
- **% Accumulated values corresponds to each possible CL code phase**
- **powerArray(1,ind) = abs(sum(signal0DC.*CLCodeSample.*sigCarr));**
- **end**
- CL code phase estimation
- **[~,acqResults.CLCodePhase(PRN)] = max(powerArray);**

acquisition.m (13/16)

➤ Acquisition implementation

- Change fine acquisition results to correspond with original sampling frequency, if downsampling is carried out.

```
.   acqResults.codePhase(PRN) = floor((codePhase - 1)/ ...  
.                                   settings.samplingFreq *oldFreq)+1;  
.   doppler = acqResults.carrFreq(PRN) - settings.IF;  
.   acqResults.carrFreq(PRN) = doppler + oldIF;
```

acquisition.m (14/16)

➤ Acquisition Results

Processing time of acquisition

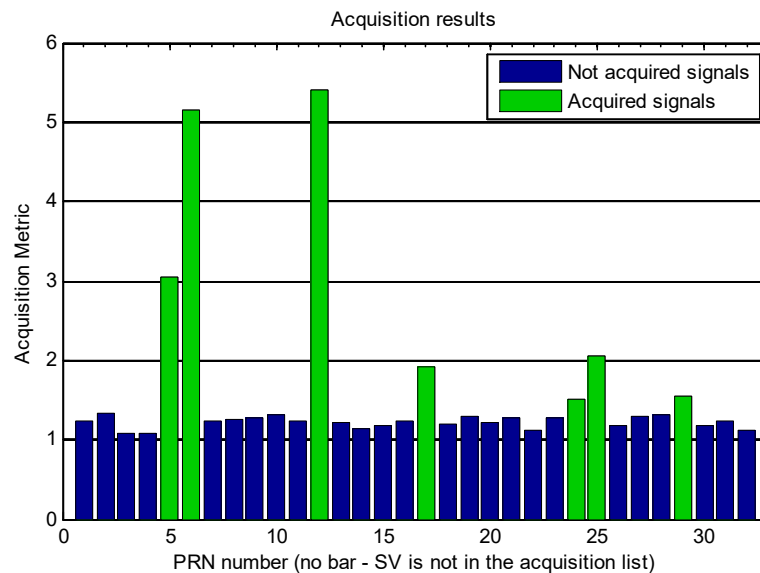
	With Downsampling*	Without Downsampling**
Elapsed time	255.8 second	2624.3 second

(*) settings.acqCohT = 10; settings.resamplingflag = 1; settings.acqSatelliteList = 1:32;

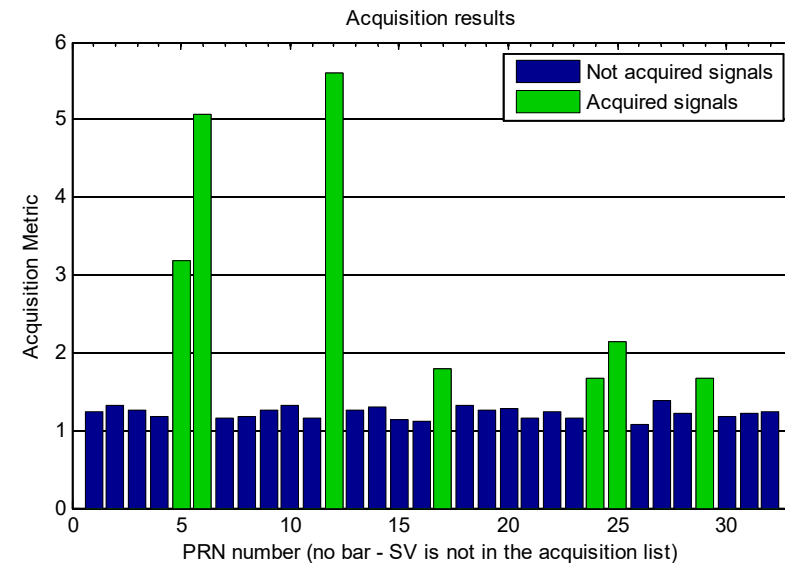
(**) settings.acqCohT = 10; settings.resamplingflag = 0; settings.acqSatelliteList = 1:32;

acquisition.m (15/16)

➤ Acquisition Results



With Downsampling*



Without Downsampling**

(*) settings.acqCohT = 10; settings.resamplingflag = 1; settings.acqSatelliteList = 1:32;

(**) settings.acqCohT = 10; settings.resamplingflag = 0; settings.acqSatelliteList = 1:32;

acquisition.m (16/16)

➤ Acquisition Results

PRN	With Downsampling*		Without Downsampling**	
	Doppler	Code offset	Doppler	Code offset
12	-1459	193394	-1463	193391
6	1033	356347	1039	356345
5	-3497	701963	-3504	701962
25	-3030	621140	-3030	621139
17	1181	879013	1190	879015
24	1534	829119	1525	829115
29	-2928	933179	-2935	933174

(*) settings.acqCohT = 10; settings.resamplingflag = 1; settings.acqSatelliteList = 1:32;

(**) settings.acqCohT = 10; settings.resamplingflag = 0; settings.acqSatelliteList = 1:32;

tracking.m (1/5)

➤ Combined tracking of data (CM) and pilot (CM)

- Combined code-tracking error estimate D

$$D = \frac{D_d + D_p}{2}$$

$$D_d = \frac{\sqrt{I_{E,d}^2 + Q_{E,d}^2} - \sqrt{I_{L,d}^2 + Q_{L,d}^2}}{\sqrt{I_{E,d}^2 + Q_{E,d}^2} + \sqrt{I_{L,d}^2 + Q_{L,d}^2}} \quad D_p = \frac{\sqrt{I_{E,p}^2 + Q_{E,p}^2} - \sqrt{I_{L,p}^2 + Q_{L,p}^2}}{\sqrt{I_{E,p}^2 + Q_{E,p}^2} + \sqrt{I_{L,p}^2 + Q_{L,p}^2}}$$

where, D_d and D_p are code tracking errors estimated from data and pilot signals, respectively.

- Combined carrier-tracking error estimate φ

$$\varphi = \frac{\varphi_d + \varphi_p}{2}$$
$$\varphi_d = \tan^{-1} \left(\frac{Q_d}{I_d} \right) \quad \varphi_p = \tan^{-1} \left(\frac{Q_p}{I_p} \right)$$

where, φ_d and φ_p are carrier tracking errors estimated from data and pilot signals, respectively.

tracking.m (1/5)

➤ Combined tracking implementation

- Pilot signal tracking

- (1) Generate CL code sequences

```
.   if (settings.pilotTRKflag == 1)
.       ...
.       CLCode = generateCLcode(channel(channelNr).PRN,settings);
.       ...
.   end
```

- (2) Generate local early, prompt and late codes

```
.   if (settings.pilotTRKflag == 1)
.       earlyCodeCL = CLCode(tcode2+settings.codeLength *(CLCodePhase-1));
.   end
.   ...
```

- (3) Generate the six standard accumulated values

```
.   if (settings.pilotTRKflag == 1)
.       I_ECL = sum(earlyCodeCL .* iBasebandSignal);
.       ...
.       Q_LCL = sum(earlyCodeCL .* qBasebandSignal);
.   end
```

tracking.m (1/5)

➤ Combined tracking implementation

- Pilot signal tracking

- (4) Combined estimation of carrier tracking error

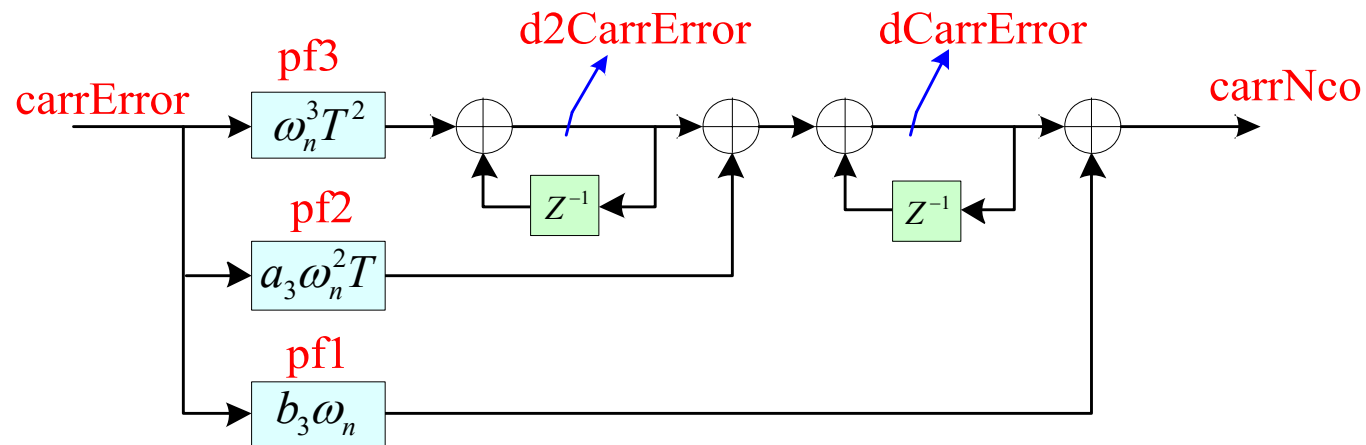
```
.   if (settings.pilotTRKflag == 1)
.       % Code tracking error of pilot signal
.       carrErrorCL = atan(Q_PCL/I_PCL)/(2.0 * pi);
.       % Combined code tracking error
.       carrError = (carrError + carrErrorCL)/2;
.   end
```

- (5) Combined estimation of code tracking error

```
.   if (settings.pilotTRKflag == 1)
.       % Carrier tracking error of pilot signal
.       codeErrorCL = ...;
.       % Combined carrier tracking error
.       codeError = (codeError + codeErrorCL)/2;
.   end
```

tracking.m (2/4)

- A 3-order PLL was used to improve dynamic performance of carrier tracking



tracking.m (3/5)

➤ PLL filter coefficients calculation

- `[pf3,pf2,pf1] = calcLoopCoefCarr(settings);`

➤ NCO update

- `d2CarrError = d2CarrError + carrError * pf3;`
- `dCarrError = d2CarrError + carrError * pf2 + dCarrError;`
- `carrNco = dCarrError + carrError * pf1;`

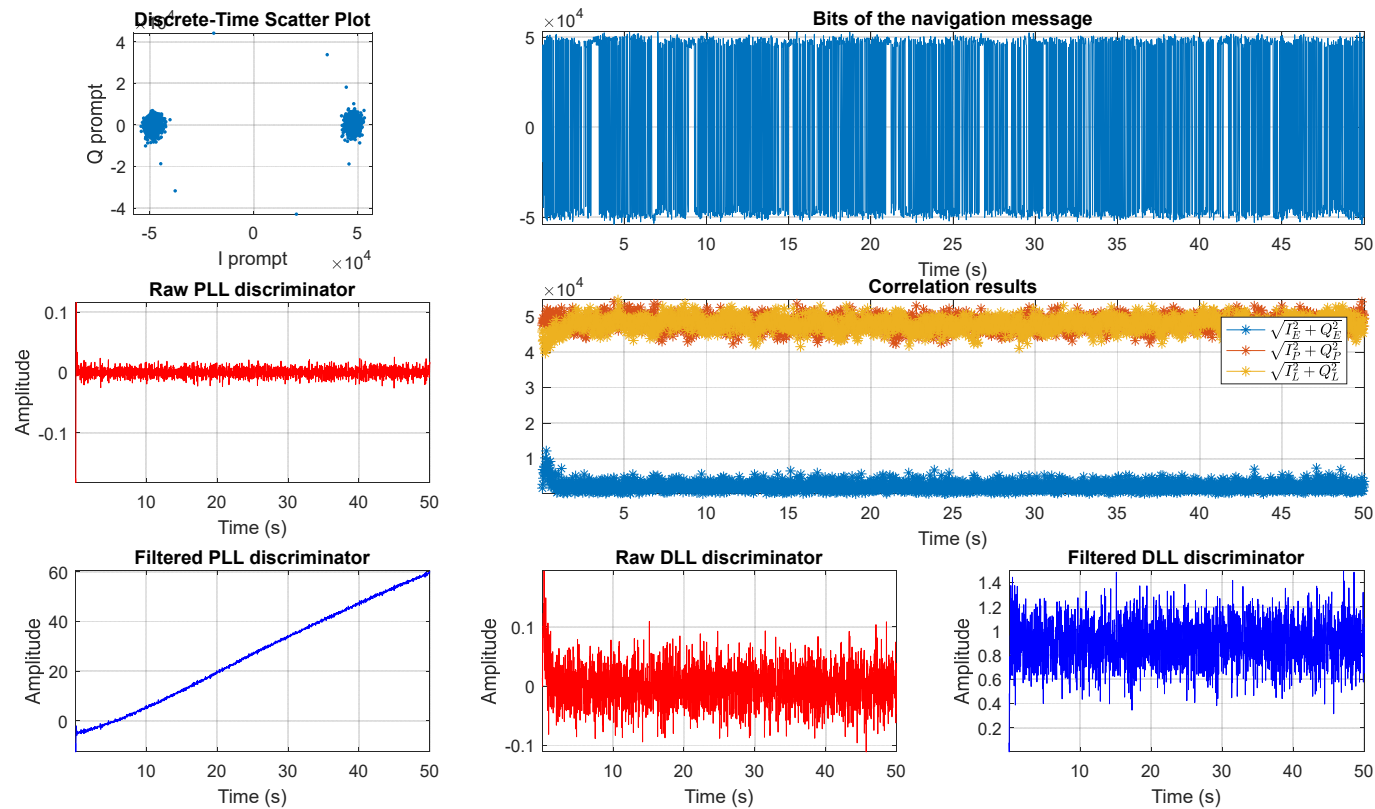
➤ Save remCodePhase for pseudorange calculation

- `trackResults(channelNr).remCodePhase(loopCnt) = remCodePhase;`

tracking.m (4/5)

➤ Tracking results

- Both PLL and DLL work well.



PRN # 9

postNavigation.m (1/2)

➤ Main changes

- **CNAV data and ephemeris decoding**
 - See “CNAVdecoding.m” and “ephemeris.m”
- **Calculation of signal transmitting time**
 - See “calculatePseudoranges.m”
- **Calculation of pseudorange and rate**
 - See “calculatePseudoranges.m”
- **Position computation for SVs**
 - See “satpos.m”

postNavigation.m (2/2)

➤ Main changes

- Start and end of measurement points

- Measurement point begins at the latest start of first messages of all channels (**firstSubFrame**), and consider Doppler smoothing
- **for channelNr = activeChnList**
- **sampleStart(channelNr) = ...**
- **trackResults(channelNr).absoluteSample(firstSubFrame(channelNr));**
- **sampleEnd(channelNr) = trackResults(channelNr).absoluteSample(end);**
- **End**
- **...**

- Correct and save local time

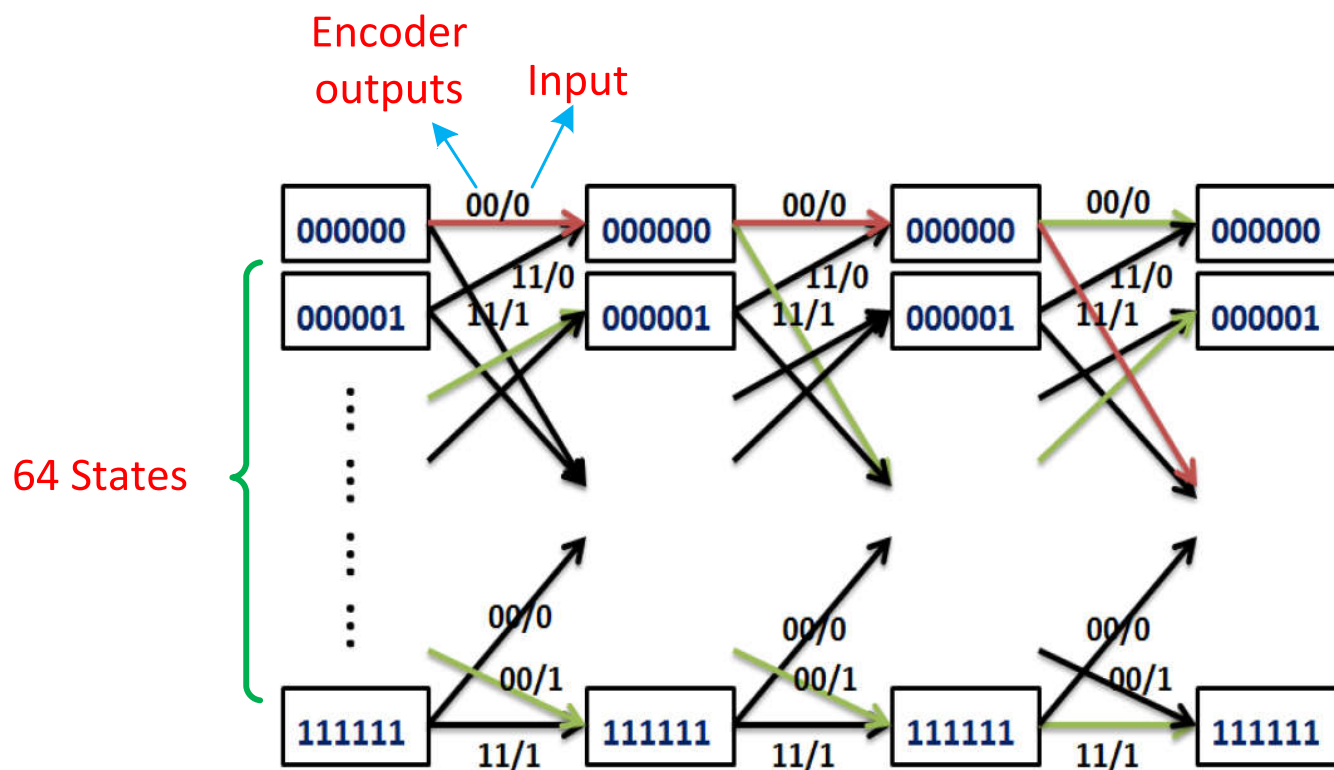
- **localTime = localTime - xyzdt(4)/settings.c;**
- **navSolutions.localTime(currMeasNr) = localTime;**

- Update local time

- Except first time, local time is updated by measurement step
- **localTime = localTime + measSampleStep/settings.samplingFreq ;**

CNAVdecoding.m (1/4)

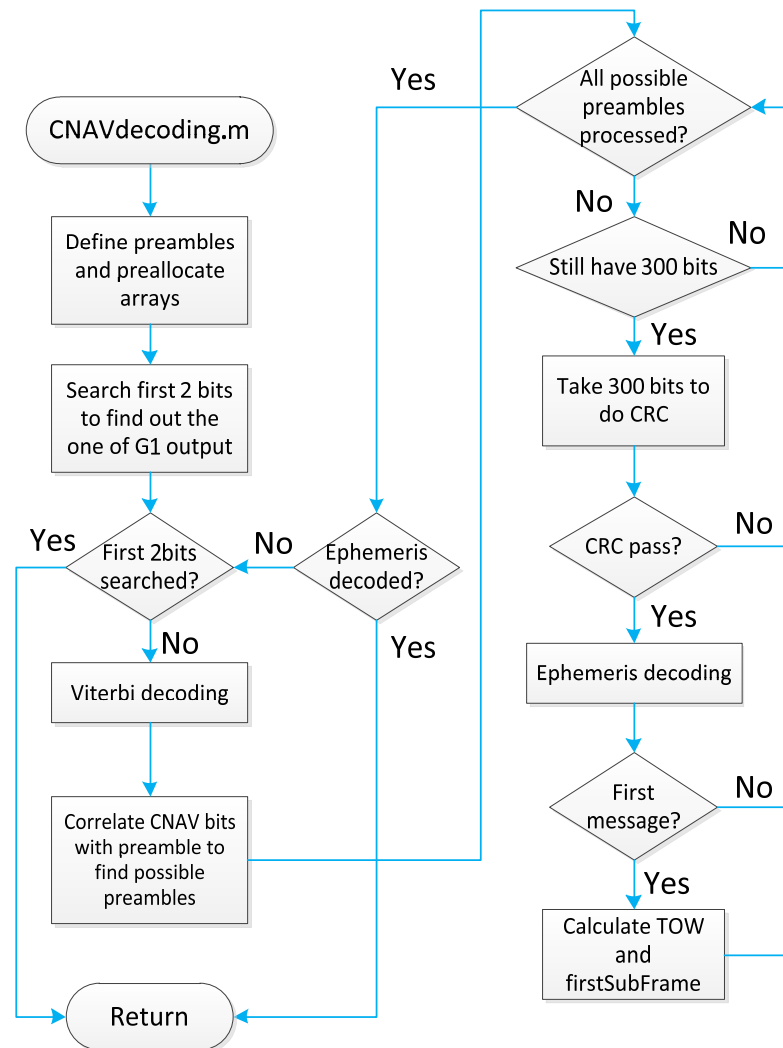
➤ Viterbi decoder algorithm



- ◆ MATLAB function: **decoded = vitdec(code,trellis,tblen,opmode,dectype)**

CNAVdecoding.m (2/4)

➤ Implementation procedure



CNAVdecoding.m (3/4)

➤ Implementation details

- Prepare trellis description for Viterbi decoder

- `trellis = poly2trellis(7,[171 133]);`
- - Search the first G1 output bit and do Viterbi decoding
- `for bitSync = 1:2`
- `decodedBits = vitdec(I_P_InputBits(bitSync:end-(bitSync-1)),`
- `trellis,tblen,'trunc','hard');`
- `...`
- `end`

- Find all possible preambles

- `index = find(round(abs(tlmXcorrResult(xcorrLength : xcorrLength * 2 - 1))) == 8.00)';`

- Correct polarity of the CNAV data bits

- `if(~isequal(navBits(1:8),preamble_01))`
- `navBits = not(navBits);`
- `End`

CNAVdecoding.m (4/4)

➤ Implementation details

- CRC-24Q check

- See MATLAB Communications System Toolbox for details
- `crcDet = comm.CRCDetector([24 23 18 17 14 11 10 7 6 5 4 3 1 0]);`
- `[~,frmError] = step(crcDet,navBits(1:300))`

- Ephemeris decoding and TOW

- See “ephemeris.m”

- “firstSubFrame”

- Start position of the first message in I_P stream
- `firstSubFrame = index(i) * 2 + G1orG2 - 2;`

ephemeris.m (1/1)

➤ Ephemeris decoding

- No special technique is needed.
- For more details, refer to “IS-GPS-200H”.

➤ Compute TOW of the first message

- Every message lasts 12 second.

· `TOW = bin2dec(navBitsBin(21:37)) * 6 - 12;`

calculatePseudoranges.m (1/2)

➤ Signal transmitting time of measurement points

- Use method of hardware receiver
- More straightforward and more accurate

$$t_t = TOM + \left(\frac{codePhase}{10230} + integer\ number\ of\ CM\ code \right) * 0.02$$

```
· codePhase = trackResults(channelNr).remCodePhase(index) + ...  
·             codePhaseStep * (currMeasSample - ...  
·             trackResults(channelNr).absoluteSample(index) );  
· transmitTime(channelNr) = (codePhase/settings.codeLength + index - ...  
·             subFrameStart(channelNr)) * settings.codeLength/...  
·             settings.codeFreqBasis + TOW(channelNr);
```

➤ Initialize signal receiving time(local time) t_r

- Use transmitting time and **settings.startOffset**
 - `maxTime = max(transmitTime(channelList));`
 - `localTime = maxTime + settings.startOffset/1000;`

calculatePseudoranges.m (1/2)

➤ Signal transmitting time of measurement points

- Use method of hardware receiver
- More straightforward and more accurate

$$t_t = TOW + \left(\frac{codePhase}{10230} + \text{integer number of CM code} \right) * 0.02$$

```
· codePhase = trackResults(channelNr).remCodePhase(index) + ...  
· codePhaseStep * (currMeasSample - ...  
· trackResults(channelNr).absoluteSample(index) );  
· transmitTime(channelNr) = (codePhase/settings.codeLength + index - ...  
· subFrameStart(channelNr)) * settings.codeLength/...  
· settings.codeFreqBasis + TOW(channelNr);
```

➤ Initialize signal receiving time (local time) t_r

- Use transmitting time and **settings.startOffset**
 - `maxTime = max(transmitTime(channelList));`
 - `localTime = maxTime + settings.startOffset/1000;`

calculatePseudoranges.m (2/2)

➤ Calculate pseudorange (PR)

$$PR = (t_r - t_t) * c$$

where, c - light speed

· `pseudoranges = (localTime - transmitTime) * settings.c;`

satpos.m (1/1)

➤ SV position calculation differences from L1 C/A SDR

- Semi-Major Axis at reference time

- `A_0 = A_REF + eph(prn).deltaA;`
- `A = A_0 + eph(prn).ADot * tk;`

- Mean motion difference from computed value

- `delta_n = eph(prn).delta_n_0 + 0.5 * eph(prn).delta_n_0Dot * tk;`

- Rate of right ascension

- `omegaDot = omegaDot_REF + eph(prn).delta_omegaDot;`

➤ Inter-Signal Correction depends on decoding of message type # 30 (IS-GPS-200H, P171)

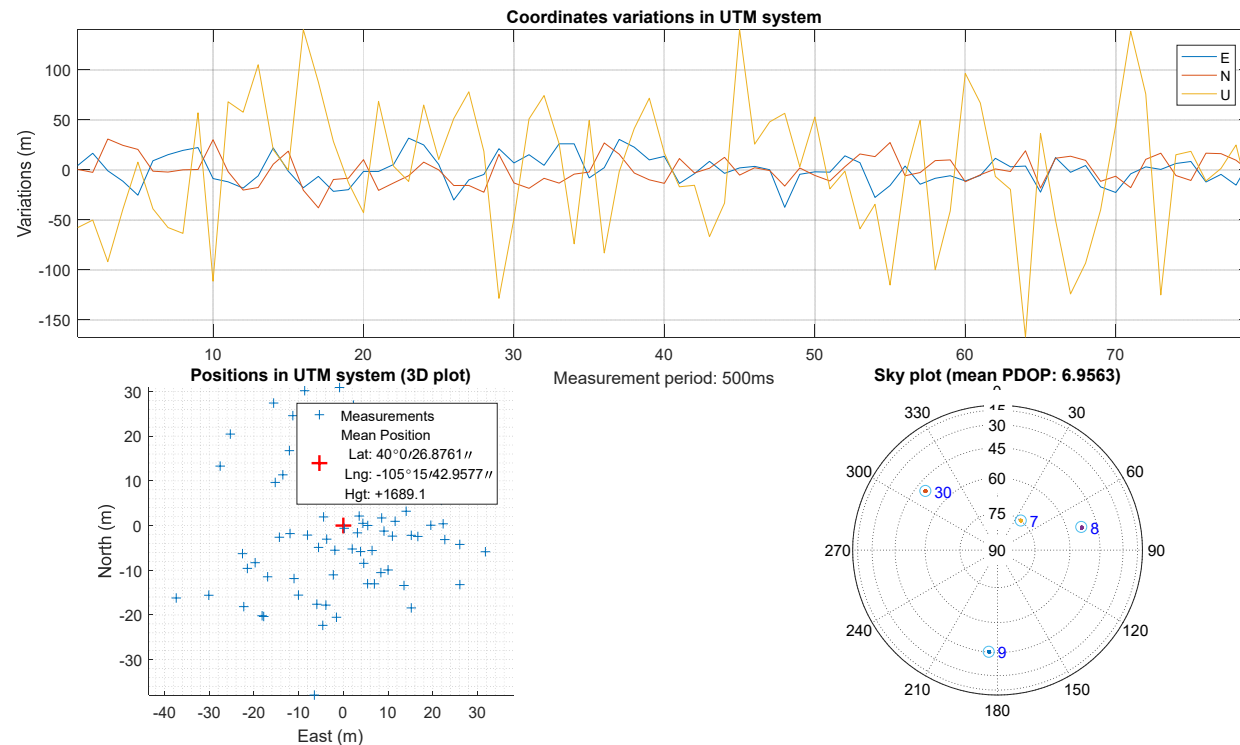
- `if(eph(prn).idValid(3) == 30)`
- `satClkCorr(satNr) = satClkCorr(satNr) - eph(prn).T_GD + eph(prn).ISC_L2C;`
- `end`

leastSquarePos.m (1/2)

➤ Receiver position calculation

- Not changed

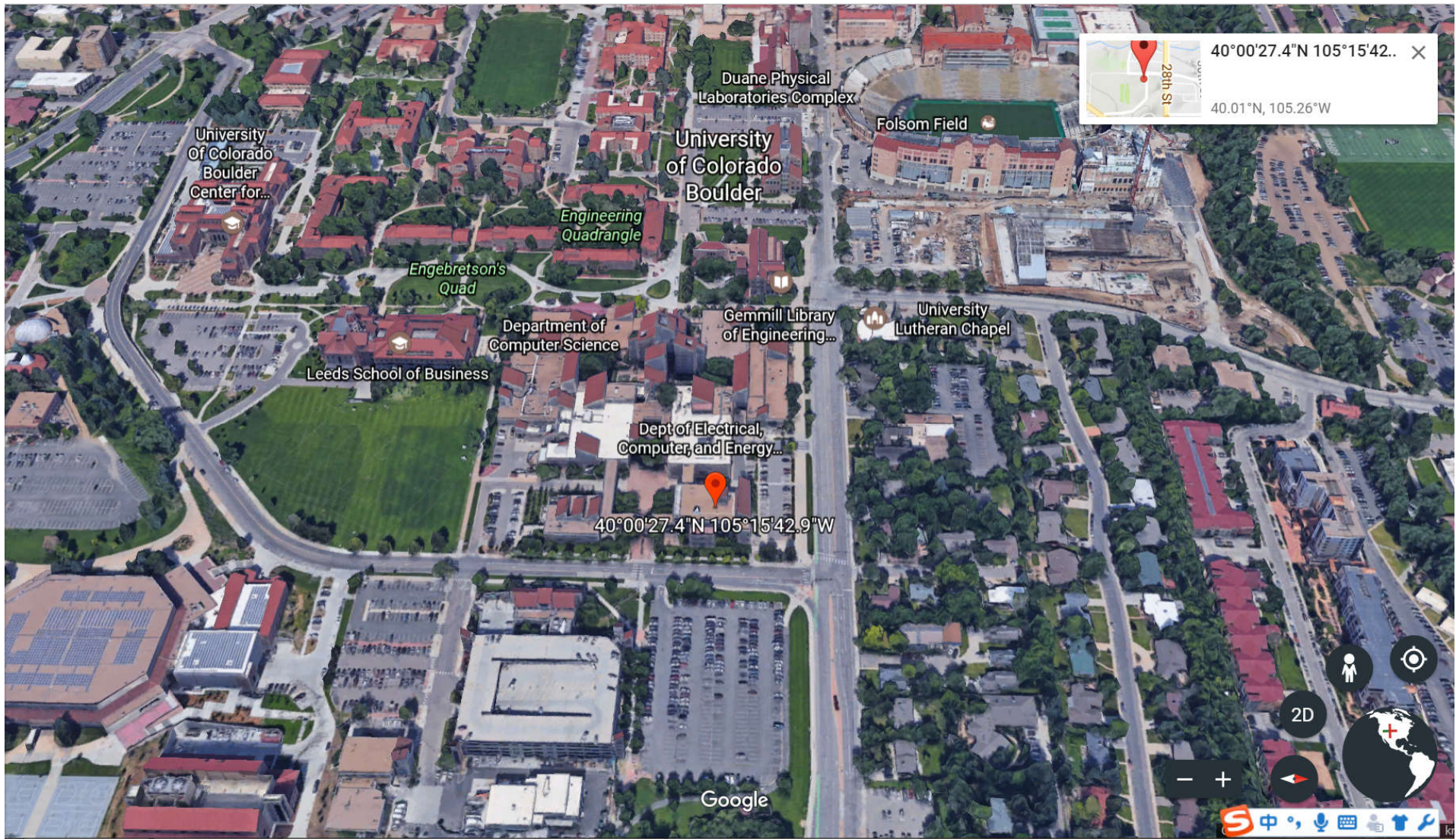
➤ Fix results



* The IF signal is collected on the roof of the Discovery Learning Center, CU-Boulder, USA.

leastSquarePos.m (2/2)

➤ Fix results



References

- **IS-GPS-200H, Navstar GPS Space Segment/Navigation User Interfaces. Sep. 24, 2013.**
- **Korvenoja P., Piche R. Efficient Satellite Orbit Approximation. Proceeding of ION GPS, Salt Lake City, UT, Sep. 19-22, 2000.**
- **Yi-Ran Sun. Generalized Bandpass Sampling Receivers for Software Defined Radio. Doctoral thesis. 2006.**
- **Hang Yin. An Adaptive Multi-frequency GPS Tracking Algorithm, GPS CNAV Message Decoding, and Performance Analysis. Master thesis. 2014.**
- **Kai, B., Akos, D. M., Bertelsen, N., Rinder, P., & Jensen, S. H.. A Software-Defined GPS and Galileo Receiver. Birkhäuser Boston, 2007.**



Thank you!

Yafeng Li lyf8118@126.com



University of Colorado
Boulder



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

